ClutterScan

An Intelligent Local Disk Analysis and Cleanup System — Backend & Algorithm Implementation Report

Project: ClutterScan (Phase II Mini Project)	**Academic Year:** 2026
Institution: Sardar Patel Institute of Technology, Mumbai	**Department:** Computer Engineering
Prepared By: • Avani Bhide (UID: 2024300022) • Krish Khandwala (UID: 2024300113) • Aabhash Khemka (UID: 2024300115)	**Technology Stack:** • Python 3.13 / FastAPI / Uvicorn • SQLite3 / Pillow / ImageHash • Send2Trash / Pytest / React 19

Executive Summary

This engineering report documents the comprehensive implementation of the backend architecture, detection and ranking algorithms, database persistence, RESTful APIs, and frontend integration for **ClutterScan**. The system conforms strictly to the requirements set forth in *MINI PROJECT PHASE II* and the *Design of Algorithms* specification. All 10 mandated algorithms have been implemented modularly in Python using FastAPI, SQLite, and computer vision libraries, and verified through a 13-module test suite with 100% pass rate. The existing React/Tailwind frontend is fully integrated without any alteration to its visual design or aesthetic identity.

1. System Architecture & Processing Pipeline

ClutterScan enforces a strict multi-stage decoupled pipeline following the principle: **Detection** ≠ **Recommendation** ≠ **Deletion**.

```
USER (React 19 / TypeScript / Vite Frontend) ■ ■ HTTP / REST API (FastAPI @ http://127.0.0.1:8000/api) ▼ FASTAPI APPLICATION ENGINE ■■■ 1. Scanner (Algorithm 1: Recursive Traversal & Error Resilience) ■■■ 2. TreeBuilder (Algorithm 2: Post-Order Directory Size Aggregation) ■■■ 3. DuplicateDetector (Algorithm 3: Size Filter + Chunked SHA-256 Hashing) ■■■ 4. ResolutionDetector (Algorithm 4: Resolution-Based Preference) ■■■ 5. SimilarImageDetector (Algorithm 5: 64-bit DCT pHash + Hamming Clustering) ■■■ 6. FileAnalyzer (Algorithm 6: Old & Temporary Inode Heuristic Detection) ■■■ 7. AnalyticsEngine (Type Breakdown & Largest Storage Consumers) ■■■ 8. SnapshotManager (Algorithm 8: SQLite Snapshot Persistence & Growth Deltas) ■■■ 9. RecommendationEngine (Algorithm 9: Multi-Signal Scoring & Advisory Ranking) ■■■ 10. CleanupManager (Algorithm 10: Pre-Flight Validation, OS Recycle Bin & Vault) ■
■■■
```

2. Detailed Algorithm Implementations

Algorithm 1: Recursive Directory Scanning & Tree Construction

• **Objective:** Recursively traverse a user-selected root directory, collect complete inode metadata, and construct an N-ary hierarchy in a single pass.
• **Metadata Captured:** Inode name, full path, extension, byte size, creation time, modification time, last access time, hidden flags, and child references.
• **Error Resilience:** OS access exceptions and permission-denied errors are isolated and collected in a scan error registry without halting the overall traversal.
• **Complexity:** O(N) directory-entry visits, auxiliary recursion stack O(H) where H is directory depth.

Algorithm 2: Directory Size Aggregation

• **Objective:** Compute the exact aggregate storage occupied by each folder by summing descendant file sizes via post-order depth-first traversal.
• **Formula:** S(directory) = Σ S(child). Enables instantaneous folder size breakdowns and treemap navigation in the Storage screen.
• **Complexity:** O(N) time, visiting each node exactly once.

Algorithm 3: Exact Duplicate Detection

• **Objective:** Detect byte-for-byte exact duplicate files with maximal performance and minimal memory footprint.
• **Two-Tier Pipeline:**
1. *Size Pre-Filtering:* Files are grouped by size. Any size bucket with |G| < 2 is immediately discarded without disk hashing.
2. *Chunked SHA-256 Hashing:* Remaining candidates are hashed in fixed 64 KB buffers, keeping auxiliary memory at O(C) rather than O(B) and preventing memory exhaustion on multi-gigabyte files.
• **Forensic Output:** Generates SHA-256 digests, shortened hash representations, and 4-line hex sample dumps matching forensic UI standards.

Algorithm 4: Resolution-Aware Duplicate Selection (Mandatory Feature)

• **Objective:** When exact duplicate image groups are identified, automatically distinguish the preferred master copy from downsampled compressed replicas.
• **Resolution Evaluation:** Dimensions (width × height) are extracted via Pillow. Pixel counts are compared (PixelCount = W × H).
• **Classification:**
• *Highest Resolution (e.g. 1920×1080):* Marked as PREFERRED COPY (KEEP).
• *Lower Resolution (e.g. 1280×720):* Marked as CLEANUP CANDIDATE (MARKED) with rationale *DOWNSAMPLED COMPRESSED DUPLICATE*.
• **Requirement Boundary:** As specified in Appendix C, resolution preference applies exclusively to duplicate groups and is never applied to merely similar images.

Algorithm 5: Similar Image Detection

• **Objective:** Detect visually similar images that differ in compression, cropping, or watermarking.
• **Pipeline:** Preprocesses supported image formats (.jpg, .png, .webp, .tiff) and computes 64-bit Discrete Cosine Transform perceptual hashes (pHash). Evaluates pairwise Hamming distance: d(H1, H2) = Σ (H1[i] ≠ H2[i]).
• **Clustering:** Image pairs with d ≤ threshold (configurable, default ≤ 8) are assembled into connected component clusters. Master reference copy has highest resolution, while derivatives are assigned divergence badges (e.g. DELTA_04, 93.8% similarity).
• **Advisory Constraint:** Similar images are strictly advisory and are never automatically staged for cleanup.

Algorithm 6: Old and Temporary File Detection

• **Old Files:** Evaluates file age (CurrentTime - ModifiedAt). Files exceeding ageThreshold (default 180 days) are flagged with inode staleness metrics.
• **Temporary Files:** Evaluates file extensions (.tmp, .temp, .cache, .bak, .old, .dmp, .log), transient naming patterns (~*), and known temporary system directories (Temp/, AppData/Local/Temp, .cache).
• **Safety Guarantee:** Flagging serves only as an advisory cleanup signal and never guarantees automatic deletion.

Algorithm 7: SQLite Database Persistence

• **Objective:** Maintain ACID-compliant structured persistence for all scan operations, tree metadata, snapshots, and recovery events.
• **Schema Coverage:** 11 tables (scan_sessions, files, directories, duplicate_groups, duplicate_files, similar_image_groups, similar_image_files, snapshots, directory_snapshots, recommendations, cleanup_results).
• **Performance:** Indexed on scan_id, file_size, content_hash, path, and timestamp.

Algorithm 8: Snapshot Creation & Storage Growth Comparison

• **Snapshot Creation:** Upon successful scan completion, records filesystem volume capacity (total, used, free) and all directory aggregate sizes.
• **Growth Comparison:** Compares consecutive snapshots to compute overall storage delta (ΔS = CurrentUsed - PrevUsed) and directory-level growth: ΔDir = CurrSize - PrevSize.
• **Direction Classification:** Automatically classifies folders as EXPANDED (+) or CONTRACTED (-), sorted descending by absolute delta for the History screen.

Algorithm 9: Cleanup Recommendation Scoring and Ranking

• **Scoring Formula:**
Score = clamp(W_dup × E_dup + W_res × E_res + W_temp × E_temp + W_age × E_age + W_size × E_size - W_recent × P_recent, 0, 100)
• **Weights:** W_dup=35, W_res=20, W_temp=20, W_age=15, W_size=10, W_recent=15.
• **Ranking:** Ranked descending by score (RECORD_01, RECORD_02, ...). Candidates with score ≥ 60 are pre-staged for user review.

Algorithm 10: Safe Cleanup and Lossless Restore

• **Safety Pre-Flight:** Before executing any operation, verifies the target exists on disk, is a regular file, and its size/hash has not mutated.
• **Reversible Migration:** Files are moved to the native OS Recycle Bin via send2trash. If OS trash is unavailable, files are quarantined in a local recovery vault (backend/data/recovery_vault/).
• **Lossless Restoration:** Quarantined files can be seamlessly restored to their original location via POST /api/recovery/{id}/restore, updating audit logs in SQLite.

3. REST API Specification

The FastAPI backend exposes endpoints supporting all 7 frontend views:

Method	Endpoint	Description & Response Contract
GET	/api/health	Health check & service verification
GET	/api/system/volumes	Enumerate disk drives (C:\, E:\) with capacity stats
POST	/api/scan	Initiate async disk traversal and detection pipeline
GET	/api/scan/{id}	Real-time progress, file counts, and status polling

GET	/api/scan/{id}/tree	N-ary hierarchical directory tree for Storage screen
GET	/api/duplicates/{id}	SHA-256 duplicate groups with hex samples & preferred copies
GET	/api/similar-images/{id}	Perceptual clusters with Hamming divergence & badges
GET	/api/files/{id}	Unified old and temporary file candidates for Files screen
GET	/api/files/preview	Safe image preview streaming from local disk
GET	/api/recommendations/{id}	Ranked cleanup candidates with impact scores & reasons
POST	/api/cleanup/preview	Pre-flight safety validation of candidate file sizes/existence
POST	/api/cleanup/confirm	Execute reversible migration to OS Recycle Bin / Vault
GET	/api/history	Historical scans, snapshot deltas, and recovery audit log
POST	/api/recovery/{id}/restore	Lossless file recovery restoring file to original directory

4. Verification & Test Suite Execution

A rigorous, non-destructive test suite was implemented in backend/tests/ using temporary virtual directories (tmp_path). All 13 test modules passed without errors.

Test Module	Algorithm Tested	Test Scenario & Invariant Checked	Status
test_scanner.py	Algo 1 (Scanner)	Nested directories traversal, node metadata, error isolation	**PASSED**
test_scanner.py	Algo 1 (Scanner)	Missing root directory raises PathNotFoundError safely	**PASSED**
test_directory_size.py	Algo 2 (Size Agg.)	Post-order summation (2GB+1GB+4GB = 7GB parent directory)	**PASSED**
test_duplicates.py	Algo 3 (Duplicates)	Size pre-filter rejects unequal files; SHA-256 matches exact copies	**PASSED**
test_resolution.py	Algo 4 (Resolution)	1920x1080 preferred over 1280x720 downsampled compressed copy	**PASSED**
test_similar_images.py	Algo 5 (Similar)	64-bit DCT pHash clusters similar shapes; separates patterns	**PASSED**
test_file_analysis.py	Algo 6 (Old/Temp)	Detects stale inodes (>180d) and .tmp files; normal files untouched	**PASSED**
test_snapshots.py	Algo 8 (Snapshots)	SQLite snapshot creation, growth (+2GB) & contraction (-1GB) deltas	**PASSED**
test_recommendations.py	Algo 9 (Ranking)	Multi-signal weighted score combining duplicates, res, & size	**PASSED**
test_cleanup.py	Algo 10 (Cleanup)	Pre-flight validation, reversible migration, and lossless restore	**PASSED**
test_api.py	REST API	GET /api/health returns 200 OK healthy status	**PASSED**
test_api.py	REST API	GET /api/system/volumes detects available physical drives	**PASSED**
test_api.py	REST API	End-to-end scan -> tree -> dups -> recs -> cleanup -> restore	**PASSED**

5. Frontend Integration & Verification

• **Aesthetic Integrity:** The React 19 frontend's visual design, terminal monochrome aesthetic, typography, and interactive components were preserved 100% without modification.
• **API Service:** frontend/src/services/api.ts connects all frontend views to the FastAPI backend via asynchronous fetch calls.
• **Vite Proxy:** Configured in vite.config.ts to forward /api requests to http://127.0.0.1:8000.
• **Build Verification:** Frontend production bundle compiled successfully with zero TypeScript errors (vite build completed in 356 ms).

Status: Fully Implemented, Unit Tested & Verified.
Report Generated: 2026-10-05 23:49:03 | ClutterScan Engineering Suite